

CSE 231: ANF Conversion

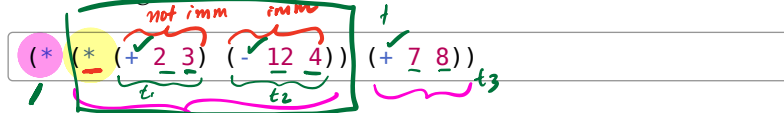
Ranjit Jhala May 18, 2026

Normal Forms

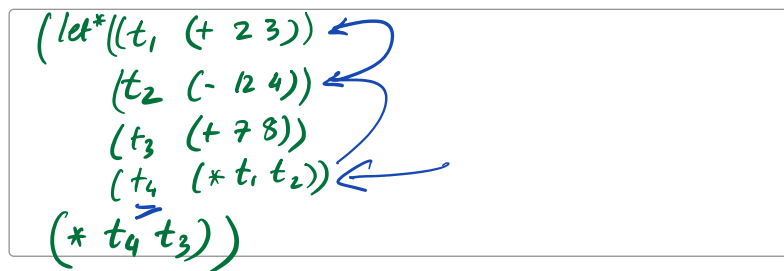
1993

Administrative Normal Form

A key step in compilation is the conversion to **A-Normal Form (ANF)** where, informally speaking, each call or primitive operation's arguments are **immediate** values, i.e. constants or variable lookups whose values can be loaded with a single machine instruction.



QUIZ: What is the A-Normal Form of the above?



Source Language

Lets start with a source language that we wish to work with:

```
type Id = String;

enum Op { Add, Sub, Mul }

enum Exp {
  Var(String),           // vars: x, y, ...
  Num(i32),              // numbers: 0, 1, ...
  Bin(Op, Box<Exp>, Box<Exp>), // binops
  Let(Id, Box<Exp>, Box<Exp>), // let-binders
  IF (Exp, Exp, Exp)
}
```

imm

(op e1 e2)

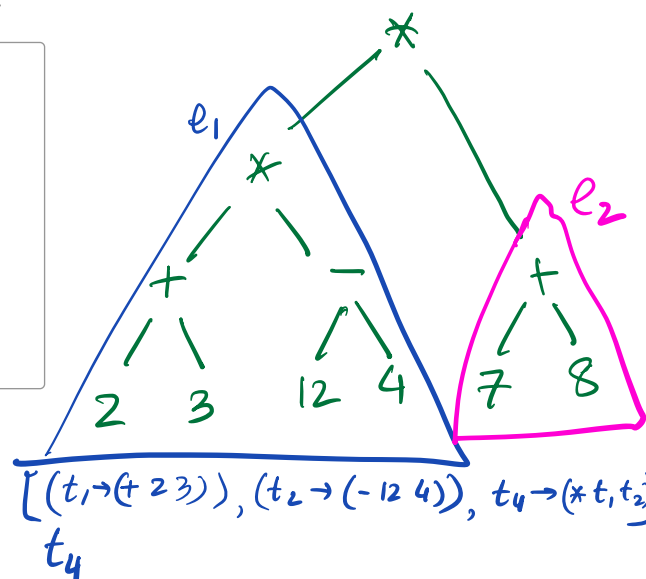
(let (x e1) e2)

$to_anf(exp) \rightarrow anf_exp$

$to_imm(expr) \rightarrow (Vec<Id, ANF>, Imm)$

For example, the source expression above corresponds to:

```
// (* (+ 2 3) (- 12 4)) (+ 7 8))
fn exp0() -> Exp {
  bin(Mul, bin(
    bin(Add, num(2), num(3)),
    bin(Sub, num(12), num(4)),
  ),
  bin(Op::Add, num(7), num(8)),
)
}
```



binds
imm
t4

$[(t_1 \rightarrow (+ 2 3)), (t_2 \rightarrow (- 12 4)), t_4 \rightarrow (* t_1 t_2)]$

Week 10

① ANF Conversion

② Type Inference } no official

08- harlequin NOT graded

immediate

- 0, 1, 2, ...
- tt, ff
- x, y, z, ...

anf
(+ imm1 imm2)

(if cond
e1
e2)

binds $[t_3 \rightarrow (+ 7 8)]$
imm t_3

A-Normal Form

Before we can describe a *conversion* to A-Normal Form (ANF), we must pin down what ANF is. Our informal description was:

Immediate Values: “constants or variable lookups whose values can be loaded with a single machine instruction”

ANF Expressions: “each call or primitive operation’s arguments are *immediate* values, i.e. constants or variable lookups”

We can encode these as Rust predicates `is_imm` and `is_anf`

```
impl Exp {
  fn is_imm(&self) -> bool {
    match self {
      - Var(_) => true, ✓
      . Num(_) => true, ✓
      Bin(_, _, _) => false, ✗
      Let(_, _, _) => false, ✗
    } IF (-, -, -) => false
  }

  fn is_anf(&self) -> bool {
    match self {
      Var(_) => true, ✓
      Num(_) => true, ✓
      Bin(_, e1, e2) => e1.is_imm() && e2.is_imm(),
      Let(_, e1, e2) => e1.is_anf() && e2.is_anf(),
    } IF (cond, e1, e2) => cond.is_anf() && e1.is_anf() && e2.is_anf()
  }
}
```

We could define brand new types, say `ImmExp` and `AnfExp`, whose values correspond to *immediate* and *ANF* terms. Unfortunately, doing so leads to a bunch of code duplication, e.g. duplicate printers for `Exp` and `AnfExp`. Try it, as an exercise.

ANF Conversion: Intuition

Recall that our goal is to convert expressions like

`( * ( * ( + 2 3 ) ( - 12 4 ) ) ( + 7 8 ) )`

into a let-bound expression of the form

```
(let (?tmp0 ( + 2 3 ))
  (let ?tmp1 = ( - 12 4 ) in
    (let ?tmp2 = ( * ?tmp0 ?tmp1 ) in
      (let ?tmp3 = ( + 7 8 ) in
        ( * ?tmp2 ?tmp3 ) ) ) ) )
```

Generalizing a bit, we want to convert

`( + e1 e2 )`

$x_1 \rightarrow \dots$

$x_2 \rightarrow \dots$

$y_1 \rightarrow \dots$

$y_2 \rightarrow \dots$

into a sequence of let-bindings that produce the names needed to make the arguments `e1` and `e2` immediate

$x_1 \rightarrow e_1'$
 $x_2 \rightarrow e_2'$

$y_1 \rightarrow \dots$
 $y_2 \rightarrow \dots$

$(+ i_1 i_2)$

$(+ e_1 e_2)$

```
(let (x1 a1) ... (let (xn an)
  (let (x1' a1') ... (let (xm' am')
    (+ v1 v2) ... )
  ) ... )
```

where $v1$ and $v2$ are immediate, and each a_i is ANF.

Forcing Arguments to be Immediate

The key requirement is a way to **force** arbitrary *argument expressions* like $e1$ into a **pair**:

- a **vector of bindings** $[(x1, a1), \dots, (xn, an)]$ where each a_i is Anf, and
- an **immediate expression** $v1$ of type Imm.

so $e1$ is equivalent to $(\text{let } (x1\ a1) \dots (\text{let } (xn\ an)\ v1))$.

Thus, we need a method to **make arguments immediate**, yielding a top-level conversion method:

```
fn to_imm(EXPR, &self, count: &mut usize, binds: &mut Vec<Id, Anf>) -> Imm;

fn to_anf(EXPR, &self, count: &mut usize) -> Anf;
```

QUIZ: Making Arguments Immediate

```
fn to_imm(&self, count: &mut usize, binds: &mut Vec<Id, Anf>) -> Imm {
  match self {
    Exp::Var(x) | Exp::Num(n) => self.clone()
    (+ e1 e2)
    Exp::Bin(op, e1, e2) => {
      let i1 = e1.to_imm(count, binds);
      let i2 = e2.to_imm(count, binds);
      let id = count.fresh();
      binds.push((id, bin(op, i1, i2)));
      Val(id)
    }
    Exp::Let(..) => {
      let a = self.to_anf(count);
      let id = count.fresh();
      binds.push((id, a));
      Val(id)
    }
  }
}
```

- **Numbers and variables** are already immediate, and are returned directly.

- **Binary operators** recursively make their operands immediate, generate a fresh binder for the operation, and return the fresh variable.
- **Let-binders** are converted to ANF and assigned to a fresh binder.

Fresh Variables

We use a count: &mut usize to produce fresh names:

```
fn fresh(count: &mut usize) -> Id {
    let name = format!("?tmp{}", count);
    *count += 1;
    name.to_string()
}
```

QUIZ: Top-Level Conversion: to_anf

```
fn to_anf(&self, count: &mut usize) -> Anf {
    match self {
        Exp::Var(x) | Exp::Num(n) => self.clone()

        Exp::Let(x, e1, e2) => { // (let (x e1) e2)
            let e1' = e1.to_anf(count);
            let e2' = e2.to_anf(count);
            let (x, e1', e2')
        }
        Exp::Bin(op, e1, e2) => { (+ e1 e2)
            let mut binds = vec![];
            { let i1 = e1.to_imm(count, &mut binds);
              let i2 = e2.to_imm(count, &mut binds);
            }
            let mut anf = Bin(op, i1, i2);
            for (id, exp) in binds.iter().rev() {
                anf = Let(id, exp, anf);
            }
            anf
        }
    }
}
```

Handwritten notes: $if(e_1, e_2, e_3) \Rightarrow \begin{matrix} e_1' = e_1.to_anf() \\ e_2' = e_2.to_anf() \\ e_3' = e_3.to_anf() \end{matrix}$ $IF(e_1', e_2', e_3')$

Handwritten diagram:

$$(* (+ 10 20) 30)$$

↓ i_2

$$[(tmp_0, (+ 10 20))] tmp_0$$

↓ i_1

binds

Handwritten derivation:

$$anf = (* tmp_0 30)$$

$$\hookrightarrow (let (tmp_0 (+ 10 20))$$

$$(* tmp_0 30))$$

In to_anf, the real work happens inside to_imm which takes an arbitrary argument expression and makes it **immediate** by generating temporary (ANF) bindings.

The resulting bindings (and immediate values) are composed by popping from the binds vector and wrapping the result with let_ to stitch them into a single Anf expression.

Handwritten diagram:

$$* (+ 10 20) (+ 30)$$

↓ i_2

$$[(tmp_0, (+ 10 20)), (tmp_1, (+ 30))] tmp_1$$

↓ i_1

binds

Handwritten example:

does NOT happen

$$if (x > 10) \quad let (t_0 (+ a b))$$

$$(+ (+ a b) 10) \quad (t_1 (* a b))$$

$$(* (* a b) 20) \quad (if (x > 10)$$

$$(+ t_0 10)$$

$$(* t_1 20))$$

|| does

Handwritten derivation:

$$anf = (* tmp_0 tmp_1)$$

$$\hookrightarrow anf = (let (tmp_1 (+ 30))$$

$$(* tmp_0 tmp_1))$$

$$\hookrightarrow anf = (let (tmp_0 (+ 10 20))$$

happen

if ($x > 10$)

(let (t_0 (+ a b))
(+ t_0 10))

(let (t_1 (* a b))
(* t_1 20))

(let (tmp₁ (+ 3 0))
(* tmp₀ tmp₁)))